

AIFX Phase 2 Manual Runbook

Hands-on guide for completing the ComfyUI face enhancement API pipeline

Prepared for Zooey Chen | Date: 2026-07-07 | Project: /Users/zooeychen/Desktop/aifx-phase1

Purpose. This runbook is designed so you do the Phase 2 steps manually first, while Codex acts as a checker, debugger, and safety net. The goal is not only to finish the feature, but also to help you remember how the API, ComfyUI workflow, LoRA mapping, and image outputs connect.

1. Current Status

- Phase 1 face detection and selected-crop saving are already working.
- Phase 2 Day 1 backend skeleton has been added and pushed to GitHub.
- The backend stores zooey.json as a fixed ComfyUI workflow template.
- The crop-only endpoint can already dry-run workflow injection without ComfyUI running.
- The ComfyUI machine is expected to be a Windows PC on LAN, probably `http://192.168.1.113:8188`.

Area	Current Value	Meaning
Mac backend API	<code>http://127.0.0.1:8000</code>	Your AIFX FastAPI service running on this Mac.
Windows ComfyUI	<code>http://192.168.1.113:8188</code>	The external ComfyUI server the Mac backend will call.
Phase 2 endpoint	<code>POST /api/v1/face-enhance</code>	Crop image in, enhanced crop image out.
Workflow template	<code>backend/workflows/zooey.json</code>	The copied ComfyUI API workflow template.
LoRA map	<code>config/lora_config.json</code>	Maps <code>character_id</code> to LoRA filename.

2. How We Will Work From Now On

- You try the next step manually first whenever possible.
- Codex explains the command, expected output, and where to look if it fails.
- Codex only edits code after you understand what the step is supposed to do.
- Every completed code change still requires README update, smoke test, commit, and GitHub sync.

3. Day 1 Manual Checklist: Real One-Crop ComfyUI Test

1. Start ComfyUI on the Windows PC: Open the Windows ComfyUI folder and start it with LAN listening enabled. The command usually looks like: `python main.py --listen 0.0.0.0 --port 8188`

2. Confirm Windows IP: On Windows, run `ipconfig` and confirm the LAN IPv4 address. If it is not `192.168.1.113`, use the real address in the next steps.

3. Test ComfyUI from the Mac: On the Mac terminal, run: `curl http://192.168.1.113:8188/system_stats`. Success means the Mac can reach the Windows ComfyUI server.

4. Start the Mac backend with COMFYUI_URL: Run: `COMFYUI_URL=http://192.168.1.113:8188 uvicorn backend.main:app --reload`

5. Check backend config: Open or curl: `http://127.0.0.1:8000/api/v1/face-enhance/config`. Confirm `comfyui_url` shows the Windows ComfyUI address.

6. Run dry-run one more time: Call `POST /api/v1/face-enhance` with `dry_run=true`. This proves image parsing, LoRA mapping, and node injection still work.

7. Run the real test: Call POST /api/v1/face-enhance with dry_run=false. This should upload one crop to ComfyUI, submit /prompt, poll /history, download the output, and save it under storage/enhanced/.

8. Inspect output: Open the enhanced image URL returned by the API. Acceptance: one enhanced cropped face image is returned without manual editing of zooey.json.

If This Fails	Likely Cause	What You Do First
curl system_stats cannot connect	Windows ComfyUI is not listening on LAN or firewall blocks 8188.	Restart ComfyUI with --listen 0.0.0.0 and allow Windows firewall access.
API returns unknown character_id	character_id not in config/lora_config.json.	Open config/lora_config.json and add the character mapping.
ComfyUI says LoRA missing	LoRA filename does not exist on the Windows ComfyUI machine.	Check ComfyUI/models/loras and match the exact filename.
ComfyUI workflow node error	zooey.json node IDs changed or custom node missing.	Open ComfyUI, install missing custom node, or export a fresh API workflow.
Timeout waiting for output	Workflow is still running, hung, or output node is different.	Check ComfyUI queue/history and confirm SaveImage node 866 is still outputting.

4. Copy-Paste Commands

Check ComfyUI from Mac

```
curl http://192.168.1.113:8188/system_stats
```

Start backend pointing to Windows ComfyUI

```
cd /Users/zooeychen/Desktop/aifx-phase1
source venv/bin/activate
COMFYUI_URL=http://192.168.1.113:8188 uvicorn backend.main:app --reload
```

Check Phase 2 config

```
curl http://127.0.0.1:8000/api/v1/face-enhance/config
```

Dry-run one crop

```
curl -X POST http://127.0.0.1:8000/api/v1/face-enhance \
-F image=@storage/crops/CaveaMan_v2-crops-01-522567e1.png \
-F character_id=cousin_sean \
-F prompt="natural face enhancement, realistic skin texture" \
-F dry_run=true
```

Real one-crop test

```
curl -X POST http://127.0.0.1:8000/api/v1/face-enhance \
-F image=@storage/crops/CaveaMan_v2-crops-01-522567e1.png \
-F character_id=cousin_sean \
-F prompt="natural face enhancement, realistic skin texture" \
-F dry_run=false
```

5. What Comes After Day 1

Day	You Manually Do	Codex Helps With	Done When
Day 2	Add more characters to lora_config.json and test API-key protected calls.	Validate mapping format, improve errors, update docs and tests.	One crop can be enhanced through protected API with a selected character_id.
Day 3	Run several selected face crops one by one and record which LoRA belongs to which face.	Implement multi-face loop and timeout/error handling.	Multiple selected faces process sequentially without crashing the server.

Day 4	Compare original, enhanced crop, and merged image visually.	Implement blending and UI comparison panel.	Final image has no obvious square crop boundary.
-------	---	---	--

6. Small Vocabulary: JSON, jsonb, jsonify, hash

Term	Plain Meaning	Where It Appears In This Project
JSON	A text format for structured data: objects, lists, strings, numbers.	zooeey.json is the ComfyUI workflow template; task metadata is also saved as JSON.
jsonb	PostgreSQL/Supabase storage type for JSON data, optimized for querying.	database/schema.sql uses jsonb for cropped_image_urls, bounding_boxes, and settings.
response.json()	Python requests method that converts an HTTP JSON response into a Python dict/list.	frontend/app.py and backend/comfyui_client.py use it to read API responses.
jsonify	A Flask helper that turns Python data into an HTTP JSON response. FastAPI usually does this automatically when returning a dict.	Not used directly in this repo. If someone says jsonify, they usually mean converting backend data to JSON response.
hash	A short fingerprint generated from data. It is often used for IDs, cache keys, or verifying content.	We use short UUID-like IDs in filenames. That is not exactly a hash, but it serves a similar anti-collision purpose.

Important distinction. zooeey.json is not code execution by itself. It is a workflow description that ComfyUI understands. Our backend deep-copies it, changes a few node inputs, then sends that JSON payload to ComfyUI /prompt.

7. Review Questions For Yourself

- Which machine runs FastAPI, and which machine runs ComfyUI?
- What does COMFYUI_URL control?
- Which node receives the uploaded crop filename?
- Where do you add a new character-to-LoRA mapping?
- What is the difference between dry_run=true and dry_run=false?